Embedded Assignments Status Report

Date: 2026-04-27 Workspace: Embedded-Assignments

Big Picture

You have 6 assignment PDFs:

- Assignment 02
- Assignment 03
- Assignment 04
- Assignment 05
- Assignment 06
- Assignment 07

You currently have 5 TA-provided project folders under `Hassabou-Assignments`:

- Assignment-02
- Assignment-03
- Assignment-05
- Assignment-06
- Assignment-07

Important gap:

- There is no `Assignment-04` folder under `Hassabou-Assignments`.

Assignment 02

Dumbed-down requirement

This assignment is not asking you to build a full RTOS. It wants you to show, using screenshots and debugging evidence, what goes wrong when the stack pointer is restored incorrectly during a context switch on Cortex-M4.

What you need to prove:

- more than one task or function exists
- a bad context switch happens because the stack pointer is wrong
- execution breaks after the bad switch
- you can show technical proof in debugger views such as registers, stack, or corrupted variables

What the instructor really wants:

- "show me that you understand why the stack pointer matters during context switching"

What the TA folder currently has

Folder: `Hassabou-Assignments/Assignment-02`

Observed contents:

- Keil project files
- `main.c`
- device/startup files in `RTE`
- generated `Objects` and `Listings`
- basic MCU headers

Cross-reference against requirements

Current state: Partial skeleton, not a finished assignment

What matches:

- there are two task functions: `Task1` and `Task2`
- there are two separate stacks: `stack_task1` and `stack_task2`
- there is a SysTick-based context-switch demo idea
- initial fake stack frames are prepared manually

What is missing or weak:

- `SysTick_Handler()` does not actually save and restore stack pointers
- there is no deliberate wrong stack-pointer restoration case yet
- there is no evidence collection mechanism in code for corruption/failure
- there is no report or screenshot guidance in the folder
- as-is, this does not yet demonstrate the failure scenario required by the PDF

Practical verdict:

- good starting idea
- not submission-ready
- needs real faulty SP-handling logic plus a debugger-driven proof plan

---

Assignment 03

Dumbed-down requirement

This assignment wants you to connect a real interrupt to a FreeRTOS task.

What you need to do:

- choose either Timer1 interrupt or GPIO interrupt
- write an ISR for it
- do not do heavy work inside the ISR
- send a signal from ISR to a FreeRTOS task
- let the task wake up and do something visible
- show scheduler and interrupt priorities are configured correctly

What the instructor really wants:

- "show me that you understand how an interrupt wakes a task in FreeRTOS"

What the TA folder currently has

Folder: `Hassabou-Assignments/Assignment-03`

Observed contents:

- Keil project files
- `main.c`
- `Required_Files` with UART/retarget helpers
- `RTE` and `inc` folders for FreeRTOS/device setup
- generated `Objects` and `Listings`

Cross-reference against requirements

Current state: Mostly aligned, likely close to usable

What matches:

- uses GPIO interrupt on Port F / SW1
- has `GPIOF_Handler()` ISR
- ISR uses `xSemaphoreGiveFromISR()`
- ISR calls `portYIELD_FROM_ISR()`
- has a FreeRTOS task `vActionTask()` waiting on the semaphore
- scheduler is started with `vTaskStartScheduler()`
- interrupt priority is explicitly configured

What is missing or weak:

- no Timer1 option, but that is fine because PDF says choose one option only
- action is just LED toggle, which is acceptable but minimal
- report content is not prepared yet
- comments/explanations are still TA-style, not personalized for your submission

Practical verdict:

- solid base
- likely easiest one to finalize first
- needs cleanup, explanation, and proof screenshots

Assignment 04

Dumbed-down requirement

This assignment is about one specific FreeRTOS macro:

- `portYIELD_FROM_ISR(xHigherPriorityTaskWoken)`

You need both theory and practice.

What you need to show:

- where the macro is defined
- what it expands to or triggers internally
- why it is needed inside an ISR
- how PendSV and interrupt priorities are involved
- practical experiment showing two cases:
  - without the macro: task switch is delayed
  - with the macro: task switch happens immediately

What the instructor really wants:

- "prove that you understand the internals, not just how to call the API"

What the TA folder currently has

There is no `Hassabou-Assignments/Assignment-04` folder.

Cross-reference against requirements

Current state: Missing

What matches:

- nothing yet in a dedicated folder

Possible reuse from nearby folders:

- `Assignment-03` already uses `portYIELD_FROM_ISR()` inside an ISR
- that makes Assignment 03 the best nearby starting point for building Assignment 04

What is missing or weak:

- no dedicated project
- no comparison case with and without the macro
- no theoretical write-up assets
- no latency/behavior measurement setup yet

Practical verdict:

- not started in TA workspace
- should probably be built by cloning/adapting Assignment 03 rather than starting from zero

---

Assignment 05

Dumbed-down requirement

This assignment wants you to prove that a Binary Semaphore can lose events.

What you need to show:

- several interrupts happen quickly while the handler task is still busy
- ISR gives a binary semaphore each time
- because binary semaphore only stores one pending signal, some interrupts are lost

- your evidence must prove that more interrupts happened than were actually handled

What the instructor really wants:

- "show me why a binary semaphore is bad for counting multiple back-to-back interrupt events"

What the TA folder currently has

Folder: `Hassabou-Assignments/Assignment-05`

Observed contents:

- Keil project files
- `main.c`
- `Required_Files` with UART/retarget helpers
- `RTE` and `inc`
- generated `Objects` and `Listings`

Cross-reference against requirements

Current state: Strong match, but still needs verification and personalization

What matches:

- uses a binary semaphore
- ISR increments `InterruptsFired`
- handler increments `TasksProcessed`
- trigger task fires the same interrupt 3 times using software trigger register
- handler task is intentionally slow, which creates the overload window
- code comments already aim at demonstrating the dropped-event behavior

What is missing or weak:

- PDF asks for technical proof that multiple interrupts fired but only one was serviced later
- code has counters, but no finished report/evidence workflow yet
- current implementation uses software-triggered interrupts; acceptable if it behaves as distinct interrupts in the simulator, but this should be validated carefully in debug mode
- code is still very close to a template and should be reworked before submission

Practical verdict:

- good candidate for completion
- probably needs only moderate modification and proof collection

---

Assignment 06

Dumbed-down requirement

This assignment is about using a Counting Semaphore like a pool of limited resources.

What you need to show:

- example: 5 tasks competing for only 2 shared resources
- counting semaphore starts at 2
- a task must take one token before using a resource
- when done, it gives the token back
- prove that no more than 2 tasks are using the resource at once
- prove the other tasks wait until a token is freed

What the instructor really wants:

- "show me that you understand counting semaphores as resource counters, not just event signals"

What the TA folder currently has

Folder: `Hassabou-Assignments/Assignment-06`

Observed contents:

- Keil project files
- `main.c`
- `Required_Files` with UART/retarget helpers
- `RTE` and `inc`
- generated `Objects` and `Listings`

Cross-reference against requirements

Current state: Conceptually aligned, but proof/report side still unfinished

What matches:

- uses `xSemaphoreCreateCounting(2, 2)`
- creates 5 worker tasks
- each worker takes semaphore, simulates usage, then gives semaphore back
- `ActiveResources` counter is present
- `TaskUsingResource[5]` array exists to show which tasks are active
- logic supports the required "only 2 at once" behavior

What is missing or weak:

- GPIO interrupt setup in this file appears unnecessary to the assignment objective
- there is no actual UART/log print output yet even though the PDF explicitly suggests evidence like terminal output or watches
- report/evidence instructions are not implemented yet
- casting integers through `void *` is common in embedded demos, but we may want a cleaner task-ID method before final submission

Practical verdict:

- very usable core logic
- needs cleanup and a better evidence strategy

Assignment 07

Dumbed-down requirement

This assignment is the "counting semaphore saves the events" version of Assignment 05.

What you need to show:

- a busy task waits on a counting semaphore
- another task triggers a real hardware interrupt 3 times in a row
- ISR gives the counting semaphore each time
- none of the events are lost
- the handler processes the queued tokens one by one later
- you must show `TokensProcessed` in Keil Logic Analyzer and the graph should jump upward like stairs

Important restriction from PDF:

- you are not allowed to fake this by calling `xSemaphoreGiveFromISR()` multiple times inside one ISR entry
- they want 3 distinct interrupt occurrences

What the instructor really wants:

- "prove that counting semaphores can queue multiple interrupt events correctly"

What the TA folder currently has

Folder: `Hassabou-Assignments/Assignment-07`

Observed contents:

- Keil project files
- `main.c`
- `Required_Files` with UART/retarget helpers
- `RTE` and `inc`
- generated `Objects` and `Listings`

Cross-reference against requirements

Current state: Strong conceptual match, but evidence path still needs to be finalized

What matches:

- uses a counting semaphore
- trigger task fires interrupt 3 times in sequence using `NVIC_SW_TRIG_R`
- ISR gives one token per ISR entry
- handler task increments `TokensProcessed`
- handler task is intentionally slow so tokens accumulate
- architecture matches the assignment objective closely

What is missing or weak:

- PDF specifically asks for Keil simulator debug run plus Logic Analyzer screenshot of `TokensProcessed`
- that screenshot/evidence workflow is not yet documented in the folder
- should verify that software-triggered Port F interrupt is accepted by the simulator as three distinct pending events for the required graph behavior
- code is still close to a template and should be individualized before submission

Practical verdict:

- closest match to the assignment PDF among all folders
- likely one of the faster assignments to finish properly

Overall Status Ranking

Best starting points:

1. Assignment 03
2. Assignment 07
3. Assignment 05
4. Assignment 06

Needs heavier work:

1. Assignment 02
2. Assignment 04

Reason:

- Assignment 03, 05, 06, and 07 already contain the right FreeRTOS building blocks.
- Assignment 07 is especially close to the PDF requirement.
- Assignment 02 has only a partial low-level demo and still lacks the actual failure demonstration.
- Assignment 04 has no dedicated project folder at all.

Recommended Work Order

1. Finalize Assignment 03 first because it is simple and already close.
2. Finalize Assignment 05 next because the binary semaphore counters are already there.
3. Finalize Assignment 07 after that because it is a natural extension of Assignment 05.
4. Finalize Assignment 06 next because the core resource-pool logic is already present.
5. Build Assignment 04 by adapting Assignment 03.
6. Finish Assignment 02 last because it needs the most low-level debugging care.

Screenshot / Report Guide Preview

This is the report strategy we should use later for each assignment after code finalization.

For every assignment report, collect these categories:

- one screenshot of the core code area proving the required mechanism exists

- one screenshot of task/variable/register behavior in debug mode
- one screenshot of the output evidence, such as Logic Analyzer, watch window, LED state logic, task states, or counters
- one short explanation under each screenshot saying what the user should notice

Per-assignment emphasis:

- Assignment 02: register view, stack pointer values, stack memory, wrong return behavior
- Assignment 03: ISR code, task waiting code, debug proof that interrupt wakes task
- Assignment 04: with/without `portYIELD_FROM_ISR`, task switch timing, PendSV explanation
- Assignment 05: `InterruptsFired` versus `TasksProcessed` counters proving dropped events
- Assignment 06: `ActiveResources` and `TaskUsingResource[]` showing max 2 active tasks only
- Assignment 07: Logic Analyzer graph for `TokensProcessed` showing staircase growth

---

Tooling Recommendation

For this phase, working here is fine because the current job is mostly:

- reading PDFs
- comparing requirements to code
- planning the implementation work
- writing status and report guidance

For the next phase, local VS Code AI is better for execution-heavy work if you want fast iteration with:

- Keil builds
- simulator runs
- debugger watches
- Logic Analyzer screenshots
- quick file-by-file edits while you inspect the project live

Best practical split:

- use this agent now for analysis, planning, report structure, and targeted code edits
- use local VS Code while actually building, simulating, debugging, and taking screenshots

That said, I can continue working here on the code changes themselves too. The main limitation here is that I cannot directly drive Keil's GUI debugger the way you can locally.